

창작 과정 기록 서비스

Trace

2인 분업 가이드 · 10주 계획

창의프로젝트1 · 2026-2학기 · 박재완 교수 · 2인 1팀

A 김용현 (맥) · B 팀원 (윈도우). Python Collector → Backend → DB → React 까지 실제로 작동시키는 것이 목표입니다. 아래 계획은 10주 · 2명 기준으로 맞췄습니다.

이 문서를 쓰는 법

1장은 같이 읽고, 2장 담당표로 역할을 나눕니다. 3장 일정표의 **관문(★)** 날짜는 **미루지 않습니다** — 기능을 줄여서라도 그 주에 끝에서 끝까지 한 번 돌립니다.

4장 계약 3개는 1주차에 정해야 둘이 서로 안 기다리고 일할 수 있습니다. 5장 결정 3개는 지금 합의해야 나중에 갈아엎지 않습니다.

1. 지금 있는 것은 하나도 안 버려집니다

이미 만들어 둔 것이 새 구조의 어디에 들어가는지입니다. 새로 배워야 하는 것은 Backend · DB · React 세 개뿐입니다.

지금 있는 것	새 구조의 어디	상태
src/collector/ (Tier 0 수집기)	Step 3 Python Collector	거의 완성 — 서버 전송만
src/core/derive.py	Step 7 Learn Mode 분석	빠대 완성
src/ui/viewer.html	React Timeline 화면	설계도로 그대로 씀
해시 체인 코드	Step 4 Proof	동작 확인됨
팀원 README 4장 데이터 계약	전 구간의 기준	가장 값진 자산

2. 분업 원칙 — 층으로 자르고, 사이에 계약을 둔다

2명이 같은 파일을 만지면 충돌하고, 한 명이 막히면 다른 한 명도 멈춥니다. “내가 만든 데이터는 내가 화면까지 그린다”로 자르는 것이 가장 안 막힙니다.

영역	A (용현·맥)	B (팀원·윈도우)
Backend · DB	●	
인증 · Work · Session API	●	

영역	A (용현·맥)	B (팀원·윈도우)
Proof 해시 체인 · sealing	●	
배포	●	
Python Collector		●
Browser Extension (Tier 1)		●
이벤트 → Learn Mode 지표		●
React 골격 · 로그인 · Dashboard	●	
React Proof Mode 화면	●	
React Learn Mode (Timeline)		●
기획 · 디자인 · 발표		●

왜 이렇게 나누나

	이유
B가 수집기를 맡는 이유	이미 전부 만들었고 이벤트 형식을 제일 잘 압니다. 윈도우라 창 제목이 읽혀 데이터가 더 풍부합니다.
A가 서버를 맡는 이유	통합 · 환경 · git 작업을 계속 해 왔습니다. Backend · 배포가 같은 성격입니다.
React를 화면 단위로 쪼개는 이유	각자 자기가 만든 데이터를 자기가 그립니다. 남에게 물어볼 일이 없어집니다.

3. 10주 일정

주	A (용현)	B (팀원)	관문
1	계약 3개 확정 + 환경 세팅	같이	
2	DB 스키마 + 이벤트 수신 API	수집기 batch 전송 붙이기	
3	받은 이벤트 DB 저장	오프라인 큐 (서버 꺼져도 안 잃음)	★ 관통 1
4	로그인 + Work / Session API	derive 로직을 서버로 이관	
5	React 골격 + 로그인 + Dashboard	Timeline 화면 (viewer → React)	
6	작업 조회 API	Timeline을 진짜 데이터로	★ 관통 2
7	해시 체인 + Root + sealing	Browser Extension (Tier 1)	
8	Proof Mode 화면	Learn Mode 지표	
9	배포 + 통합 테스트	같이	★ 관통 3
10	파일럿 테스트 + 발표	같이	

★ 관통이란

그 주 안에 끝에서 끝까지 한 번 돌려 보는 것입니다. 화면이 없어도 되고 숫자만 나와도 됩니다.

3주차 관통 1이 가장 중요합니다. “수집기에서 이벤트가 나가서 DB 에 들어갔다”만 확인하면 됩니다. 이걸 미루면 9주차에 “안 맞는다”로 2주가 날아갑니다.

4. 1주차에 반드시 정할 것 — 계약 3개

이게 있어야 둘이 서로 안 기다리고 일합니다.

계약	무엇	상태
① 이벤트 형식	수집기가 내보내는 한 줄의 모양	이미 있음 — README 4장
② 화면이 받는 형식	Timeline 이 읽는 session 데이터	이미 있음 — README 4장
③ API 형식	요청 · 응답 JSON 의 모양	이번 주에 만들어야 함

②가 이미 있어서 B 는 서버 없이도 Timeline 화면을 끝까지 만들 수 있습니다. 가짜 데이터를 넣고 만들면 됩니다.

③만 정하면 A 도 수집기 없이 서버를 만들 수 있습니다. 계약이 정해지면 “네 거 끝나야 내 거 시작”이 사라집니다.

5. 지금 결정해야 할 것 3개

① 해시 체인을 어디서 만드나 — 문서는 “서버”인데 바꾸시길 권합니다

문서 7장은 Backend 에서 체인을 만든다고 되어 있는데, 9장의 batch 전송과 충돌합니다. 로컬에 쌓여 있는 동안은 체인 보호를 못 받고, 서버가 만들면 운영자(우리)가 고칠 수 있습니다. “우리도 못 고친다”는 주장이 성립하지 않습니다.

```
collector -> make chain locally (already built)
           -> send to server
server     -> recompute & verify -> store
           -> anchor root hash (OpenTimestamps)
```

구조는 문서 그대로 두고 만드는 위치만 바꾸는 것이라 5·6장은 안 바꿉니다. 오프라인에서도 체인이 안 끊깁니다.

② 맥의 창 제목 — 문서 3·12장에 있는데 가져올 수 없습니다

	윈도우	맥
앱 이름	가능	가능
창 제목	가능	불가 — ‘화면 기록’ 권한이 필요해 쓰지 않음
클립보드 길이 · 해시	가능	가능
입력 · 삭제 횟수	가능	가능 — ‘입력 모니터링’ 권한 필요

맥에서는 브라우저가 어느 탭인지 알 수 없어, 11장 지표 중 Struggle Time · Skip Count 의 정확도가 떨어집니다. Tier 1 확장(7주차)이 붙으면 해결됩니다. 기획서에 “맥은 확장 설치 시 완전 동작”이라고 명시할지 정해야 합니다.

③ Tier 2 (VS Code 확장) — 이번 학기엔 빼는 것을 권합니다

10주에 Backend + DB + React + Collector + 확장 두 개 + 배포는 2명에서 넘칩니다. Tier 2 를 빼면 11장의 Retry Loop 지표도 같이 빠집니다 (Tier 2 데이터가 필요하다고 문서에 적혀 있습니다). 발표 자료에서 미리 빼 두는 편이 낫습니다.

	이번 학기	비고
Tier 0 (OS)	Must	거의 완성
Tier 1 (브라우저)	Should	7주차
Tier 2 (VS Code)	Won't	Retry Loop 지표도 함께 제외

6. 가장 큰 위험 둘

통합을 뒤로 미루는 것

각자 8주 동안 열심히 만들고 9주차에 붙이면 거의 확실히 안 맞습니다. 그래서 일정에 관문을 3번 박아 두었습니다. 관문 날짜는 못 미룹니다. 기능을 줄여서라도 그 주에 관통시킵니다.

React 를 늦게 시작하는 것

처음 배우면 2주는 헤맵니다. 5주차 전에 A 가 Vite + React 빈 화면 띄우기만이라도 미리 해 보기를 권합니다. 1시간이면 됩니다.

7. 이번 주에 할 일

	누가	무엇
1	같이	③ API 계약 작성 → docs/api.md 로 저장소에 올리기
2	같이	5장의 결정 3개 합의
3	A	React 빈 화면 띄워 보기 (1시간)
4	B	수집기 batch 전송 붙이기 준비

Proof 가 증명하는 것과 증명하지 않는 것

증명함	증명하지 않음
기록이 만들어진 뒤에 고쳐지지 않았다	AI 를 쓰지 않았다
기록된 이벤트의 순서	작업의 저자가 누구인지
기록된 이벤트의 시간	작업의 품질
	기록되지 않은 시간에 무엇을 했는지

이 구분은 발표에서 반드시 나옵니다. 두 사람이 같은 문장으로 말할 수 있어야 합니다.